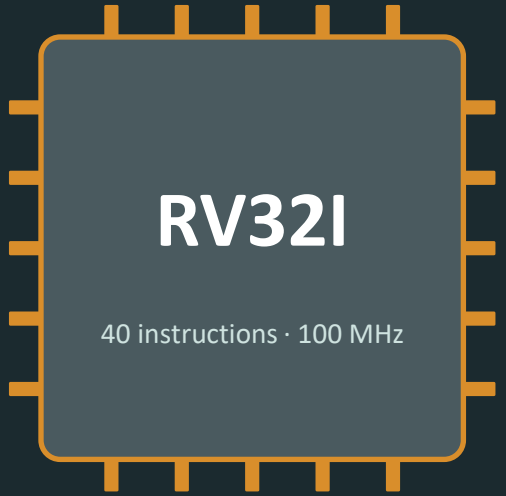
Design and FPGA Implementation of an RV32I RISC-V Microprocessor using Vitis HLS

From C++ to a working CPU on the Zybo board



Nishanthini Sivakumar
Work-Based Learning (WBL), NIELIT Chennai

What we will cover

1. **Basics**
   Microprocessor, RISC-V, registers, instruction formats, all 40 instructions

2. **Architecture**
   Datapath, instruction cycle, memories, multi-cycle design

3. **Design in HLS**
   C++ code structure, HLS pragmas, bugs fixed

4. **Verification**
   RARS, test programs, C-sim, co-simulation, synthesis

5. **FPGA**
   Vivado block design, constraints, timing, problems solved

6. **Demo + results**
   Bubble sort on the board with VIO, final results

What is a microprocessor?

	Microprocessor (this project)	Microcontroller
Contains	CPU core (+ memory for the program)	CPU + memory + peripherals
Peripherals	None	GPIO, UART, timers, ADC …
Job	Fetch, decode and execute instructions	Control a device on its own
Examples	Intel Core, ARM Cortex-A, our RV32I CPU	Arduino (AVR), STM32, ESP32

In one sentence

A microprocessor is a chip that reads instructions from memory one by one and executes them: add, compare, load, store, jump. Our project builds this CPU core and its instruction and data memory, without any peripherals.

What is RISC-V and why RV32I?

Term	Meaning
ISA	Instruction Set Architecture: the list of instructions a CPU understands and how they are encoded
RISC	Reduced Instruction Set Computer: few, simple, fixed-size instructions
RISC-V	An open, free ISA from UC Berkeley (2010). No licence fee, used in industry and education
RV32I	The 32-bit base integer set: 32 registers, 32-bit instructions, 40 instructions
Extensions	M (multiply), A (atomic), F/D (floating point), C (compressed): added on top of RV32I

32 registers

32-bit instructions

40 instructions

6 formats

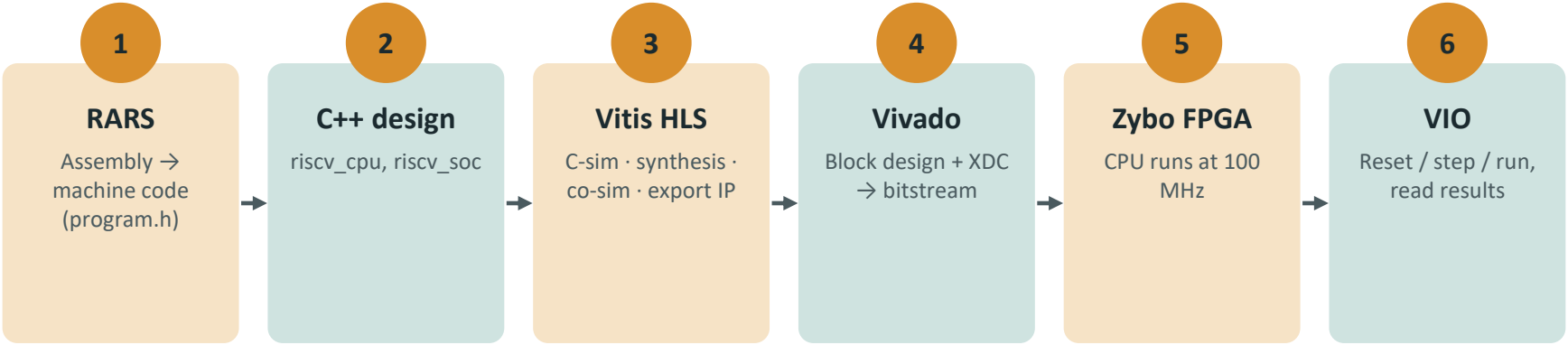
Project goal and tools

Tool	Used for
C++	Writing the processor as a program (behaviour)
Vitis HLS 2022.1	C simulation, converting C++ to Verilog (synthesis), co-simulation, IP export
RARS	RISC-V assembler + simulator: machine code and the correct (golden) results
Vivado 2022.1	Block design, constraints, implementation, bitstream, Hardware Manager
VIO	Virtual Input/Output: buttons and displays inside the FPGA, used from the PC
Zybo board	Zynq-7010 FPGA (xc7z010clg400-1), 125 MHz board clock

Goal

Build a complete RV32I processor in C++, convert it to hardware with HLS, verify every instruction against RARS, and show it running on a real FPGA, with step-by-step control and register/memory inspection using VIO.

Complete project flow



Verification at every step

The same program runs in RARS (the correct answer) and on our CPU: in C simulation, in RTL co-simulation, and on the FPGA. **If the registers and memory match RARS, the CPU is correct.**

1

RISC-V basics

Registers, instruction formats and all 40 RV32I instructions

32 registers: x0 to x31

Register	ABI name	Usual use
x0	zero	Always 0. Writes are ignored
x1	ra	Return address (function call)
x2	sp	Stack pointer
x3, x4	gp, tp	Global / thread pointer
x5–x7, x28–x31	t0–t6	Temporary values
x8, x9, x18–x27	s0–s11	Saved values
x10–x17	a0–a7	Function arguments / results (a7 = system call number)

Why x0 = 0?

A constant zero is very useful: mv, li, nop and compare-with-zero are all done with x0. In hardware: write_enable = (rd != 0).

In our demo

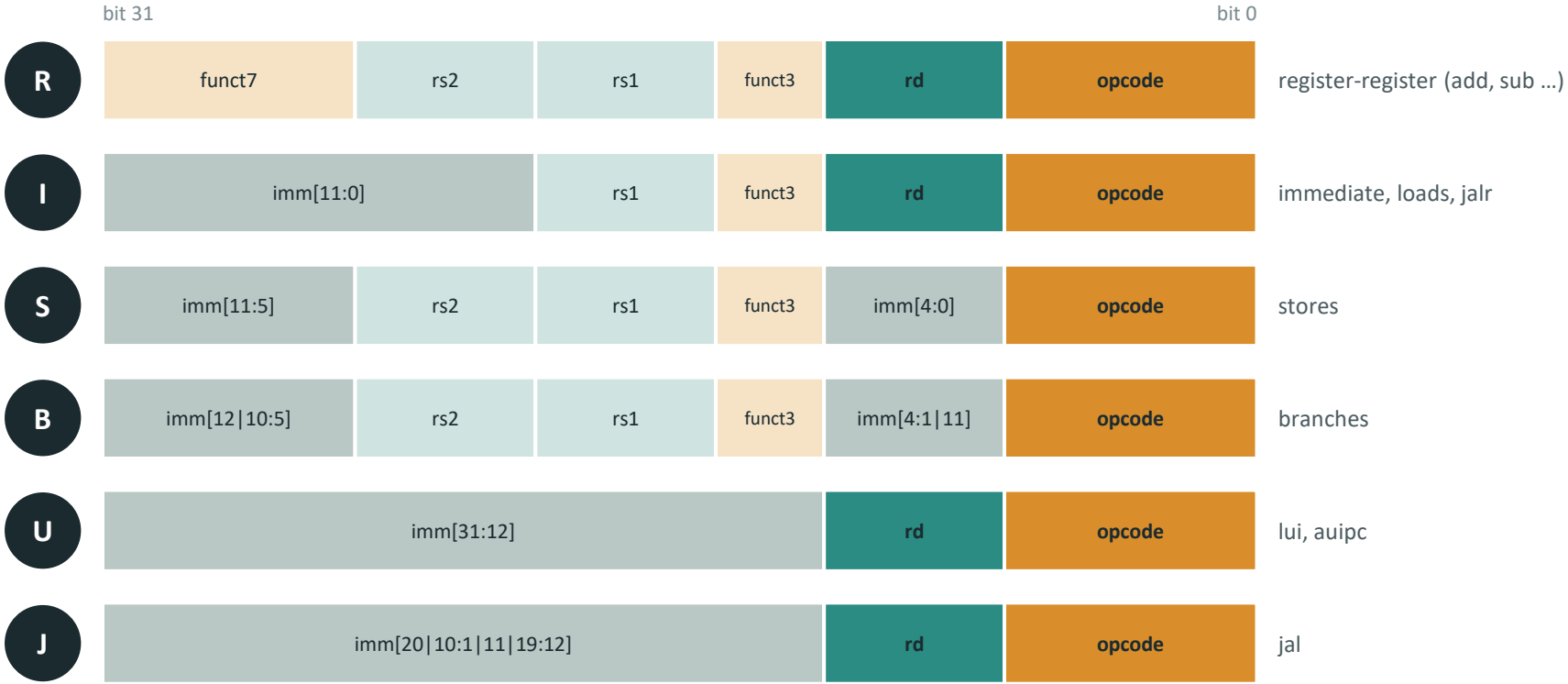
x10 (a0) = array address 0x2000

x11 (a1) = n = 6

x12–x14 = loop counters

x17 (a7) = 10 = exit code

The 6 instruction formats (32 bits each)



Instructions 1/4: R-type (register–register)

#	Mnemonic	Full name	Operation
1	ADD	Add	rd = rs1 + rs2
2	SUB	Subtract	rd = rs1 − rs2
3	SLL	Shift Left Logical	rd = rs1 << rs2[4:0]
4	SLT	Set Less Than	rd = (rs1 < rs2) ? 1 : 0 (signed)
5	SLTU	Set Less Than Unsigned	rd = (rs1 < rs2) ? 1 : 0 (unsigned)
6	XOR	Exclusive OR	rd = rs1 ^ rs2
7	SRL	Shift Right Logical	rd = rs1 >> rs2[4:0] (fill with 0)
8	SRA	Shift Right Arithmetic	rd = rs1 >> rs2[4:0] (fill with sign bit)
9	OR	OR	rd = rs1 | rs2
10	AND	AND	rd = rs1 & rs2

Instructions 2/4: I-type immediate and U-type

#	Mnemonic	Full name	Operation
11	ADDI	Add Immediate	rd = rs1 + imm
12	SLTI	Set Less Than Immediate	rd = (rs1 < imm) ? 1 : 0 (signed)
13	SLTIU	Set Less Than Immediate Unsigned	rd = (rs1 < imm) ? 1 : 0 (unsigned)
14	XORI	XOR Immediate	rd = rs1 ^ imm
15	ORI	OR Immediate	rd = rs1 | imm
16	ANDI	AND Immediate	rd = rs1 & imm
17	SLLI	Shift Left Logical Immediate	rd = rs1 << shamt
18	SRLI	Shift Right Logical Immediate	rd = rs1 >> shamt (0 fill)
19	SRAI	Shift Right Arithmetic Immediate	rd = rs1 >> shamt (sign fill)
20	LUI	Load Upper Immediate	rd = imm << 12
21	AUIPC	Add Upper Immediate to PC	rd = PC + (imm << 12)

Instructions 3/4: loads (I-type) and stores (S-type)

#	Mnemonic	Full name	Operation
22	LB	Load Byte	rd = sign-extend(mem8[rs1 + imm])
23	LH	Load Halfword	rd = sign-extend(mem16[rs1 + imm])
24	LW	Load Word	rd = mem32[rs1 + imm]
25	LBU	Load Byte Unsigned	rd = zero-extend(mem8[rs1 + imm])
26	LHU	Load Halfword Unsigned	rd = zero-extend(mem16[rs1 + imm])
27	SB	Store Byte	mem8[rs1 + imm] = rs2[7:0]
28	SH	Store Halfword	mem16[rs1 + imm] = rs2[15:0]
29	SW	Store Word	mem32[rs1 + imm] = rs2

Byte = 8, half = 16, word = 32 bits

Address = rs1 + imm.

Signed load of byte 0xF6 gives 0xFFFFFFF6 (−10).

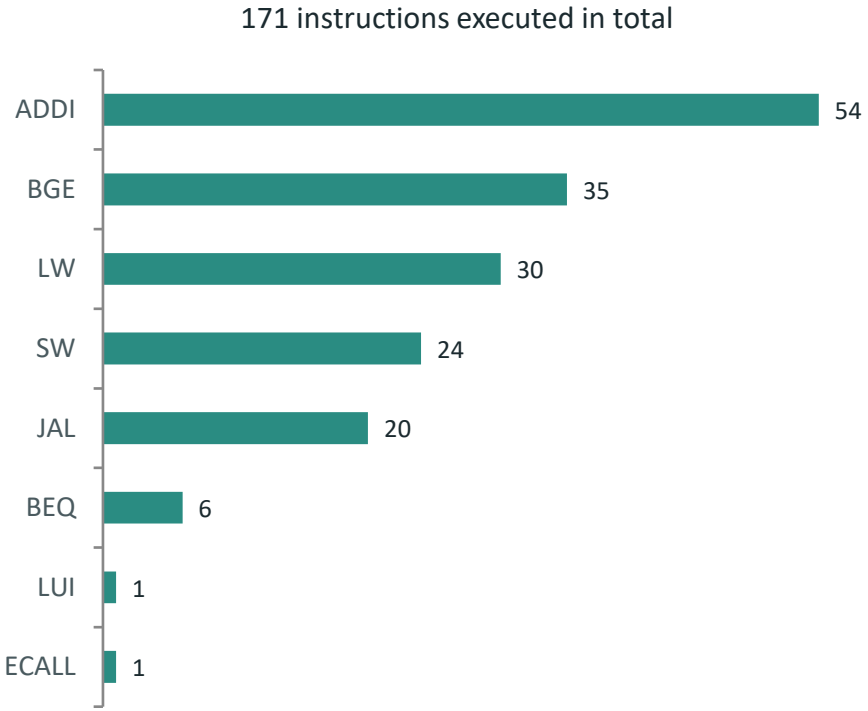
Unsigned load gives 0x000000F6 (246).

RISC-V is a load/store architecture: only these 8 instructions touch memory.

Instructions 4/4: branches, jumps and system

#	Mnemonic	Full name	Operation
30	BEQ	Branch if Equal	if (rs1 == rs2) PC = PC + imm
31	BNE	Branch if Not Equal	if (rs1 != rs2) PC = PC + imm
32	BLT	Branch if Less Than	if (rs1 < rs2) PC = PC + imm (signed)
33	BGE	Branch if Greater or Equal	if (rs1 >= rs2) PC = PC + imm (signed)
34	BLTU	Branch if Less Than Unsigned	if (rs1 < rs2) PC = PC + imm (unsigned)
35	BGEU	Branch if Greater or Equal Unsigned	if (rs1 >= rs2) PC = PC + imm (unsigned)
36	JAL	Jump And Link	rd = PC + 4; PC = PC + imm
37	JALR	Jump And Link Register	rd = PC + 4; PC = (rs1 + imm) & ~1
38	FENCE	Fence (memory ordering)	No operation in our single-core CPU
39	ECALL	Environment Call	System call. Our CPU: halt (stop)
40	EBREAK	Environment Break	Debugger breakpoint. Our CPU: halt (stop)

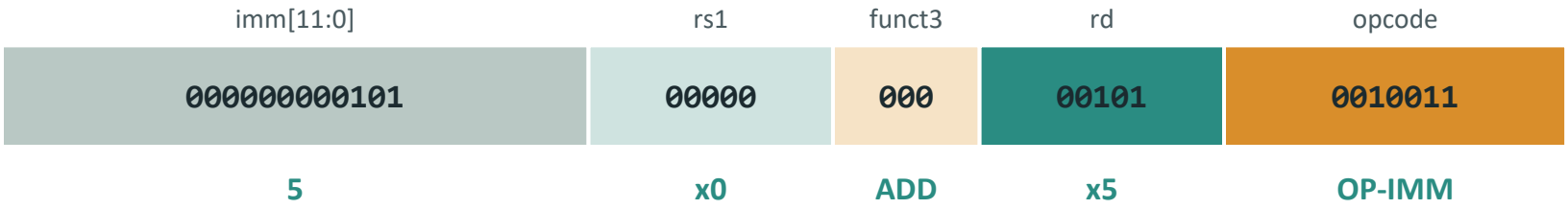
Instructions executed in the demo (bubble sort)



Instr.	Used for
ADDI	load numbers, counters, pointer + 4
BGE	loop end, and the signed compare
LW	read two neighbours
SW	write input array, swap
JAL	jump back to loop start
BEQ	all passes done?
LUI	x10 = 0x2000
ECALL	stop the CPU

8 different instructions · 31 in the program · 171 executed (loops repeat)

Example: decoding one instruction — 0x00500293



Step	What the decoder sees	Decision
1	opcode = 0010011	I-type arithmetic (OP-IMM)
2	funct3 = 000	ADDI
3	rs1 = 0, rd = 5, imm = 5	addi x5, x0, 5
4	Execute	x5 = x0 + 5 = 0 + 5 = 5

Check in RARS

Address 0x00000008

Code 0x00500293

Basic: addi x5, x0, 5

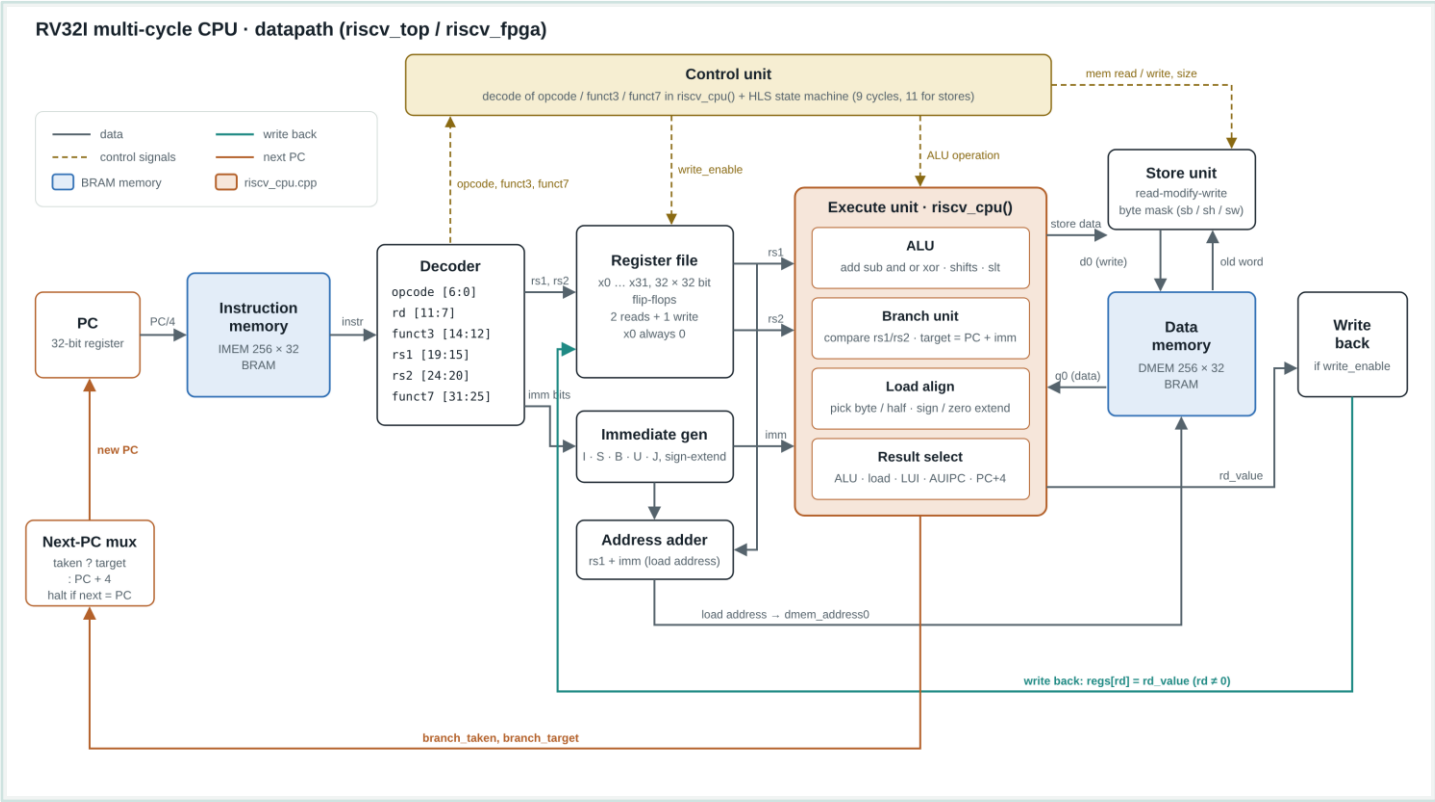
This is the 3rd instruction of the demo program.

2

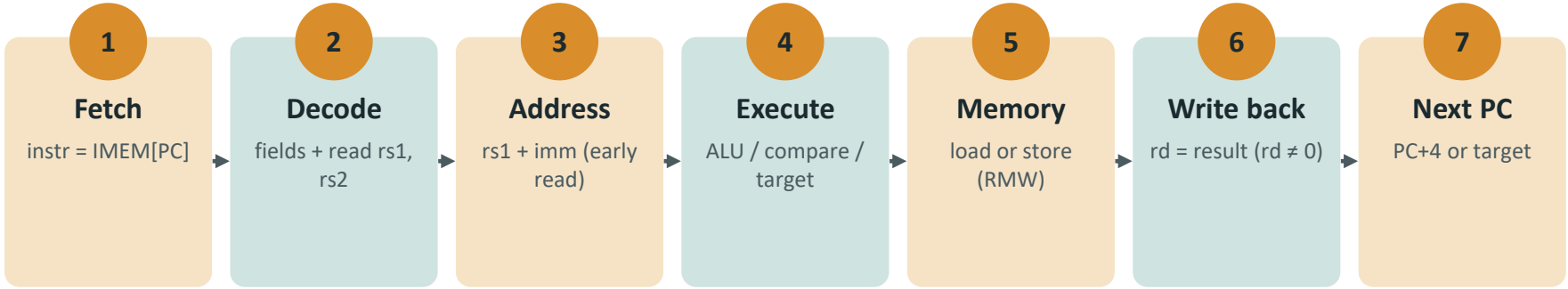
CPU architecture

Datapath, instruction cycle, memories and timing

Datapath of the RV32I CPU



One instruction cycle: 7 steps



Instruction type	Clock cycles	Time at 100 MHz
Most instructions (ALU, load, branch, jump)	9	90 ns
Stores (read-modify-write)	11	110 ns
Average CPI (measured in co-simulation)	≈ 9.3	≈ 93 ns
Demo: 171 instructions	≈ 1,590	≈ 16 µs

CPI

Cycles Per Instruction. Each step needs block-RAM reads and logic, so one instruction takes several clock cycles.

Memory: Harvard architecture

	Harvard (our design)	Von Neumann
Memories	Separate instruction memory and data memory	One shared memory for both
Access	Fetch and data access can happen at the same time	One at a time (bottleneck)
Our sizes	IMEM 256 × 32 bit (1 KB, ROM) DMEM 256 × 32 bit (1 KB, RAM)	—
FPGA	Both become block RAM (2 BRAMs)	—

Address in program (RARS)	Where it is in our CPU
0x00000000 – 0x000003FF (.text)	IMEM word = PC >> 2
0x00002000 – 0x000023FF (.data)	DMEM word = (addr >> 2) & 255 → 0x2000 = word 0

RARS setting: Settings → Memory Configuration → "Compact, Text at Address 0"

Single-cycle vs multi-cycle vs pipelined

	Single-cycle	Multi-cycle (our design)	Pipelined
Idea	Whole instruction in 1 long clock cycle	Each instruction uses several short cycles	5 instructions overlap, like an assembly line
CPI	1	≈ 9.3	≈ 1 (ideal)
Clock	Slow (longest path)	100 MHz	Fast
Hardware	Simple	Simple, one datapath	Complex: hazards, forwarding, stalls
Why chosen / not	Not possible with BRAM read latency	Simple and correct; easy to verify	Future work

In HLS

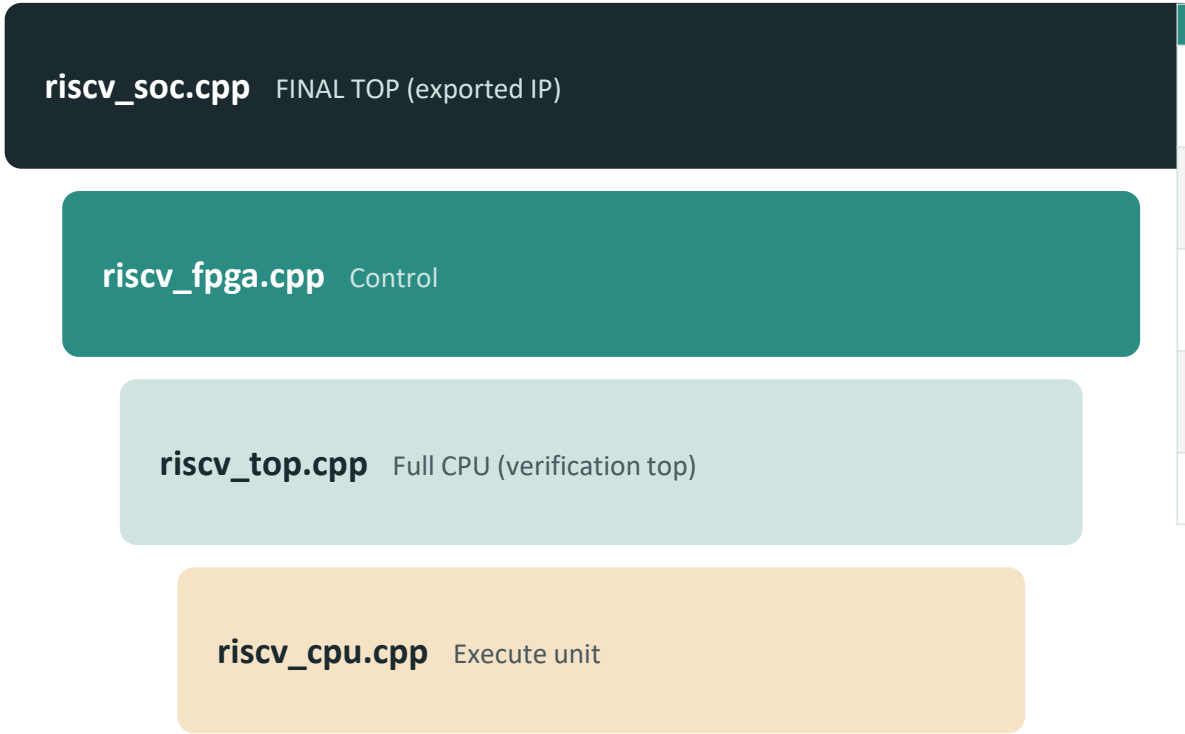
#pragma HLS PIPELINE off on the main loop: the PC and the registers are needed by the next instruction (loop-carried dependency), so pipelining caused II violations.

3

Design in Vitis HLS

From C++ functions to a hardware IP block

Code structure: 4 layers



File	Contains
riscv_soc.cpp	IMEM ROM from program.h, DMEM RAM, VIO ports, dbg_mem
riscv_fpga.cpp	Static PC + 32 registers, reset / step / run buttons (rising edge), debug outputs
riscv_top.cpp	Fetch, register read, memory, write back, next PC, halt
riscv_cpu.cpp	Decode + execute all 40 instructions (ALU, branch, load/store)
riscv_cpu.h / program.h	Prototypes, memory sizes / machine code

Important HLS pragmas and settings

Pragma / setting	Where	Why
#pragma HLS PIPELINE off	Main execution loop	Multi-cycle CPU; avoids II violations from the PC/register dependency
#pragma HLS ARRAY_PARTITION complete	regs[32]	32 separate registers, so rs1 and rs2 can be read in the same cycle
#pragma HLS UNROLL	Reset loop	Clear all 32 registers at once
#pragma HLS INTERFACE ap_none	dbg_pc, dbg_reg …	Plain wires (no handshake), so VIO can read them at any time
static variables	PC, regs, dmem	Keep their value between calls → flip-flops / BRAM
ap_start tied to 1 (Vivado)	Block design	The IP runs again and again, so it always reacts to the buttons
Target 10 ns, part xc7z010clg400-1	Solution settings	100 MHz on the Zybo FPGA

Bugs found and fixed

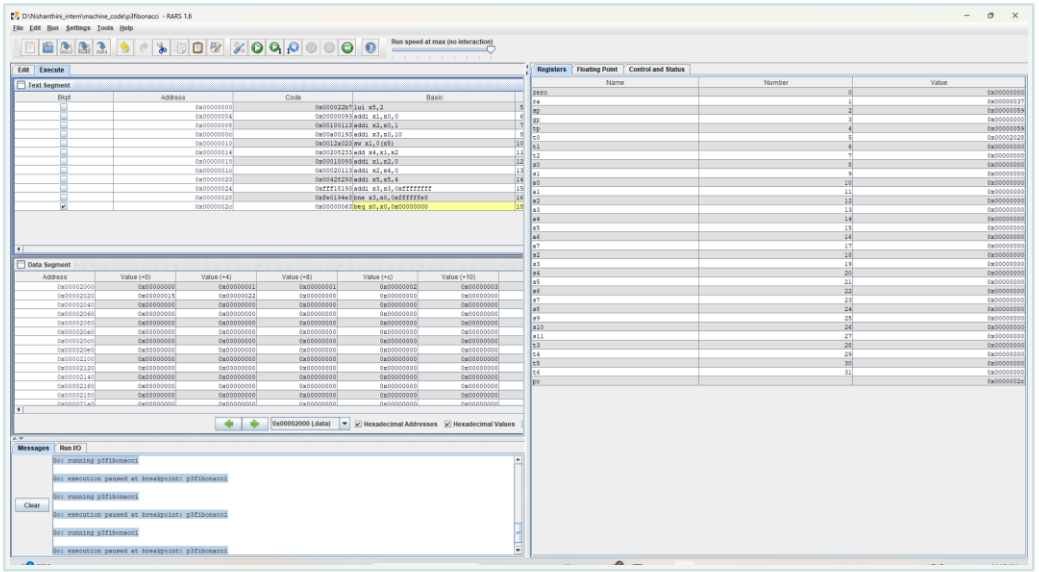
#	Problem	Effect	Fix
1	Trailing comma in the port list	Did not compile	Removed
2	Load code outside the opcode block	Wrong results for non-load instructions	Moved inside the load case
3	JALR after a wrong else	JALR never executed	Fixed the if / else chain
4	LUI/AUIPC/JAL/JALR wrote rd without checking rd ≠ 0	x0 could change	write_enable = (rd != 0)
5	Byte/half lane not selected by addr[1:0]	Wrong byte loaded	Shift by addr[1:0] × 8
6	Auto-pipelining of the RUN loop	II violation warnings	PIPELINE off
7	Wrong clock pin in XDC (K17 instead of L16)	No clock, VIO not detected	Board is original Zybo → L16

4

Verification

RARS, test programs, C simulation, co-simulation and synthesis

Why RARS? Assembler + golden reference



Job 1: assembler

Assembly (addi x5, x0, 5) → machine code (0x00500293). Dump → make_program_h.py → program.h → instruction memory.

Job 2: golden reference

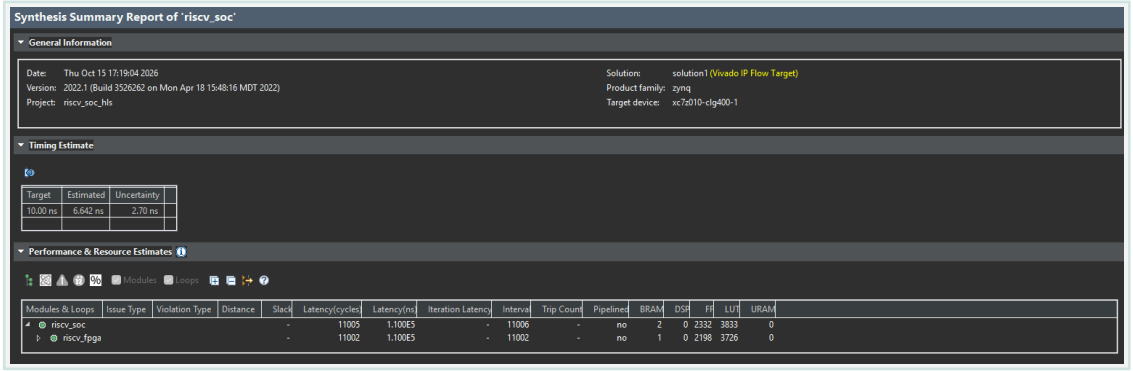
RARS runs the same program and shows the correct registers and memory. Our CPU must give the same values.

Test programs and verification levels

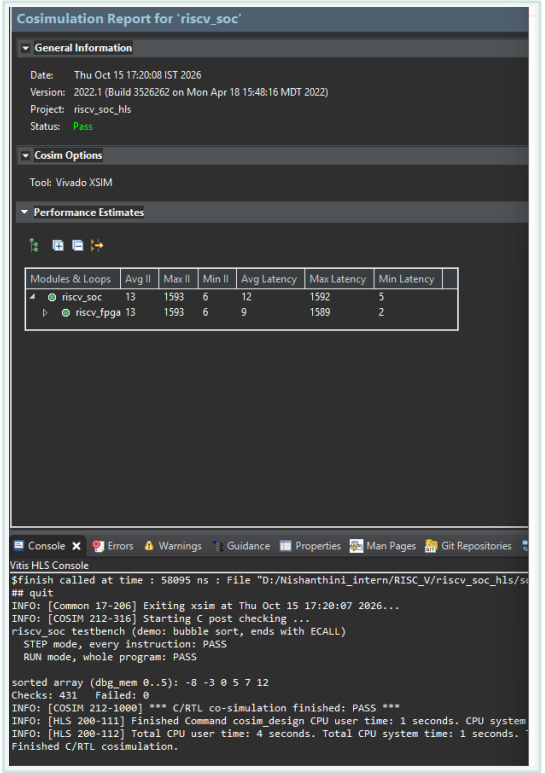
Program	What it tests
program1	Smoke test: lui, addi, add, sw, lw
program2	Loop, sb/lb/lbu/sh/lhu, function call jal/jalr
p3_fibonacci	First 10 Fibonacci numbers to memory
p4_bubblesort	Nested loops, signed compare, swap
p5_all_instructions	Every RV32I instruction, results stored in memory
demo_bubblesort	Bubble sort + ECALL (runs on the FPGA)

Level	Result
Unit test (riscv_cpu)	32 / 32 PASS
2 million random instructions vs reference	0 mismatches
500 random programs vs ISS	0 mismatches
C-sim, 5 programs vs RARS	91 / 91 PASS
riscv_fpga (step / run)	1,775 / 1,775 PASS
riscv_soc final IP	431 / 431 PASS
C/RTL co-simulation	PASS
FPGA board (VIO vs RARS)	All values match

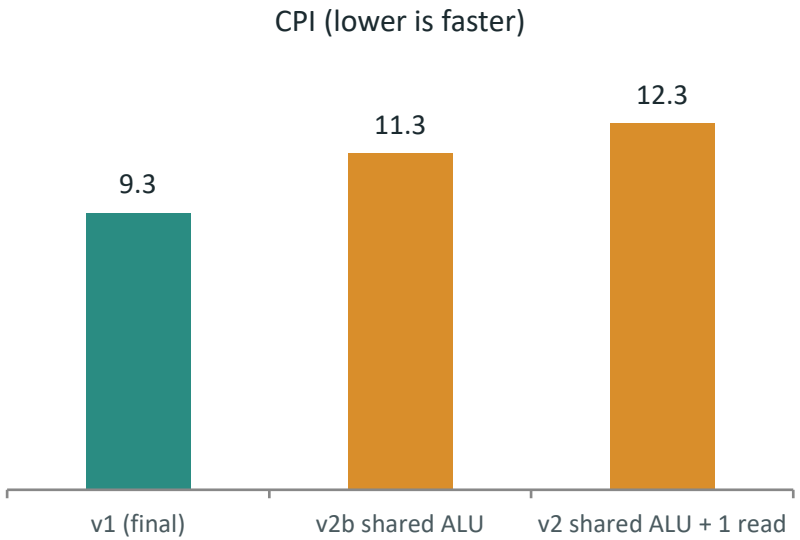
HLS results: synthesis and co-simulation



Result	Value
Estimated clock	6.642 ns (target 10 ns)
Estimated Fmax	150.55 MHz
LUT / FF	3,833 / 2,332
BRAM / DSP	2 / 0
Co-simulation	PASS, 431 checks



Optimization: shared ALU (v2) vs v1



Version	LUT	FF	ns	CPI
v1 (final)	4,037	2,102	6.705	9.3
v2b	3,708	1,790	7.131	11.3
v2	3,776	1,792	7.131	12.3

Decision: keep v1

Sharing one adder saved ~8% LUTs but made the CPU 22–32% slower: in v1 all units compute in parallel and a mux picks the result.

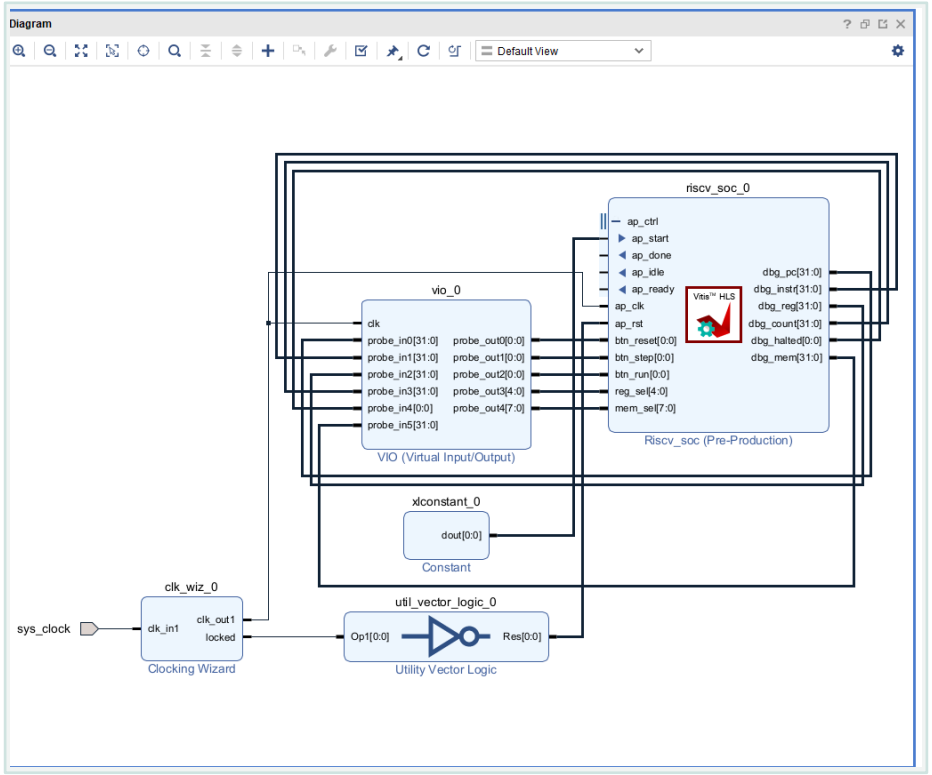
Lesson: change one thing at a time (v2b isolated the shared ALU).

5

FPGA implementation

Vivado block design, constraints, timing and bring-up

Vivado block design



Block	Job
clk_wiz_0	125 MHz board clock → 100 MHz clk_out1; locked = clock stable
util_vector_logic (NOT)	ap_rst = NOT locked: CPU held in reset until the clock is stable
xlconstant (1)	ap_start = 1: IP runs continuously
riscv_soc_0	Our CPU IP with IMEM + DMEM
vio_0	5 outputs: btn_reset, btn_step, btn_run, reg_sel, mem_sel 6 inputs: dbg_pc, instr, reg, count, halted, mem

Constraints, timing and the clock-pin problem

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	0.283 ns	Worst Hold Slack (WHS):	0.025 ns	Worst Pulse Width Slack (WPWS):	2.000 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	8325	Total Number of Endpoints:	8309	Total Number of Endpoints:	4050

All user specified timing constraints are met.

Item	Value
Clock constraint	125 MHz on sys_clock (period 8 ns)
CPU clock	100 MHz
WNS (setup)	+0.283 ns ✓
WHS (hold)	+0.025 ns ✓
Failing endpoints	0

Board	125 MHz clock pin
Zybo Z7-10	K17
Original Zybo (ours)	**L16**

Symptom: "debug hub core was not detected", no VIO.
LED test: LD0 = counter bit (blinks if clock runs), LD1 = locked. Both off → no clock → wrong pin. Fixed with L16.

6

Demo on the board

Bubble sort on the RV32I CPU, controlled with VIO

Bubble sort: compare neighbours, swap

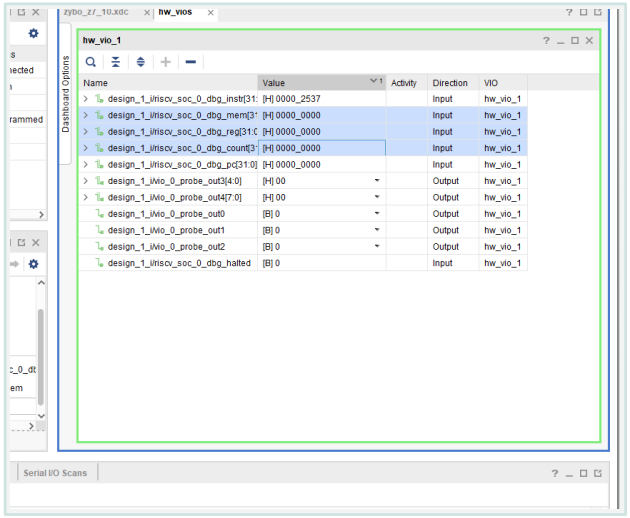
Pass	Compares	Array after the pass	Fixed at the end
Start	—	[5, −3, 12, 0, 7, −8]	—
1	5	[−3, 5, 0, 7, −8, 12]	12
2	4	[−3, 0, 5, −8, 7, 12]	7
3	3	[−3, 0, −8, 5, 7, 12]	5
4	2	[−3, −8, 0, 5, 7, 12]	0
5	1	[−8, −3, 0, 5, 7, 12]	sorted ✓

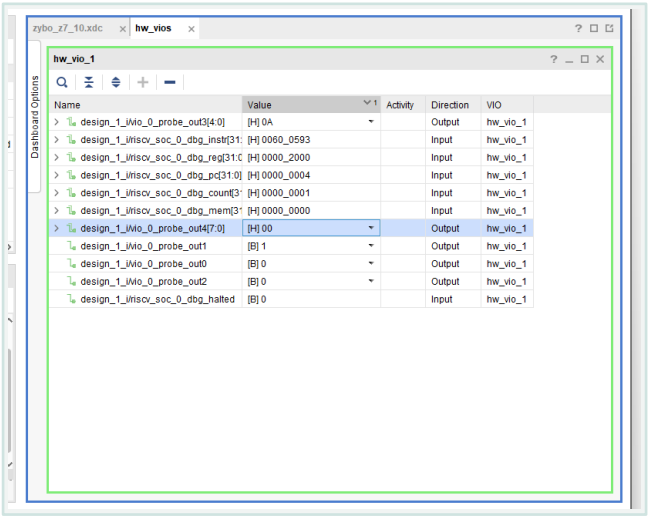
Step	Instructions
Write input array	addi + sw (×6)
Load neighbours	lw x6, 0(x14) · lw x7, 4(x14)
In order?	bge x7, x6, noswap
Swap	sw x7, 0(x14) · sw x6, 4(x14)
Loops	addi, bge, beq, jal
Stop	addi x17, x0, 10 · ecall

6 numbers → 5 passes · 15 compares · 10 swaps · 171 instructions

The input numbers are inside the program (addi + sw), so RARS gives one exact expected answer.

VIO: reset and step mode

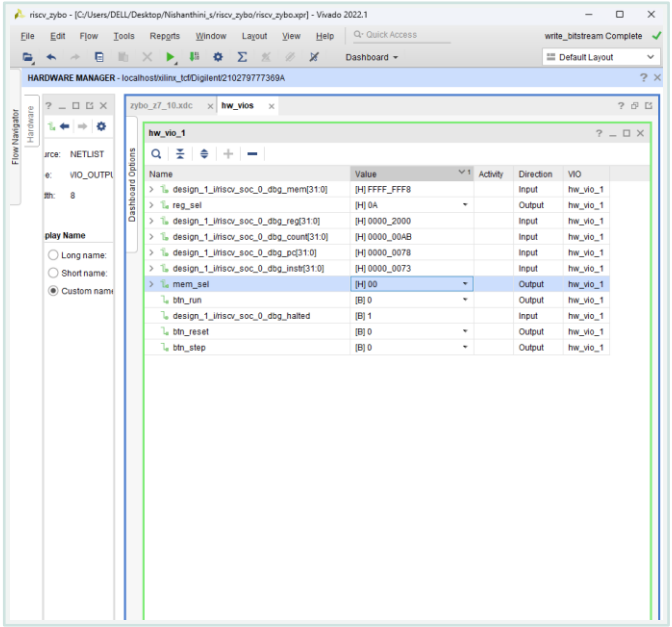




After reset	Value
dbg_pc	0000_0000
dbg_instr	0000_2537 = lui x10, 2
dbg_count	0

After 1 step	Value
dbg_pc	0000_0004
x10 (reg_sel = 0A)	0000_2000 ✓ (RARS a0)
dbg_instr	0060_0593 = addi x11, x0, 6

VIO: run mode — the sorted array matches RARS



Signal	FPGA (VIO)	RARS	
halted / count	1 / 0x00AB (171)	finished	✓
pc, instr	0x78, ecall	0x78 ecall	✓
mem 0x2000	FFFF_FFF8 (−8)	−8	✓
mem 0x2004	FFFF_FFFD (−3)	−3	✓
mem 0x2008	0000_0000 (0)	0	✓
mem 0x200C	0000_0005 (5)	5	✓
mem 0x2010	0000_0007 (7)	7	✓
mem 0x2014	0000_000C (12)	12	✓
x10 / x17	0x2000 / 0xA	a0 / a7	✓

Negative numbers use two's complement: −8 = 0xFFFFFFF8

Final results

40 / 40

RV32I instructions

100 MHz

on Zybo, timing met

CPI ≈ 9.3

multi-cycle

3.8K / 2.3K

LUT / FF (HLS)

2 BRAM

0 DSP

All PASS

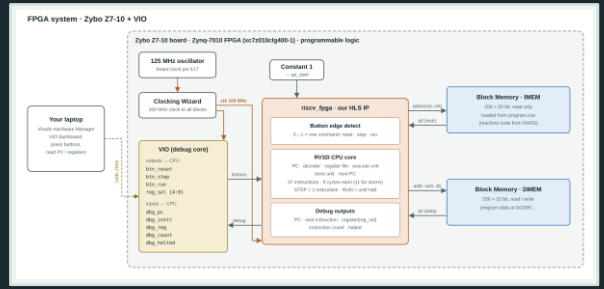
sim · co-sim · board

Lessons learned and future work

Lessons learned	Future work
Verify against a trusted golden model (RARS) at every stage	5-stage pipeline (CPI → ~1) with hazard handling
Change one variable at a time in an experiment	Run-time input: VIO write port into data memory
Read board documentation: the clock pin differs between Zybo and Zybo Z7	Peripherals (GPIO, UART, timer) → microcontroller
Simple LED tests quickly find clock and reset problems	M extension (multiply / divide)
HLS pragmas control the hardware: pipeline, partition, interfaces	Run p5_all_instructions on the board

Thank You

RV32I RISC-V microprocessor · Vitis HLS · Zybo FPGA



Nishanthini Sivakumar
Work-Based Learning (WBL), NIELIT Chennai